



JAVA PROGRAMMING (340)

STATE – 2019

Production Portion:

Program 1: Team Directory _____ (305 points)

TOTAL POINTS _____ (***305 points***)

**Graders: Please double check and verify all scores
and answer keys!**

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.



You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The input file will contain only ASCII characters.
- A test input file will be available and named, "team.txt."

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must use Javadoc comments.

Note to Graders: The contestants have been given a file named team.txt. You must first delete that file from the contestant's flash drive and run the program for Test case #1. This should result in a message that the file was not found. Then replace the file on their flash drive with the grader's file with the same name and run Test case #2. You'll run their program with a grader's file named team.txt which will contain different data than the student file. This will produce different results than the student's file. You must copy the grader's team.txt file into the folder where their program runs. Listed below are the test cases and their results.

Test case #1 verifies that the program checks for the correct input file. The output wording does *not* need to be exact.

Text case #2 processes the correct data file name and produces output. The output wording *must* be exact.

Test Case #1

The input file was not found

Test Case #2

[Rita Stevens, Adele, Sue Woods, Barbara, Zeb]

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____	10 points
The projects main class is named Sort	_____	10 points
The projects helper class is named TeamMember	_____	10 points

Program Execution

The program runs from the USB flash drive	_____	15 points
---	-------	-----------

If the program does *not* execute, then the remaining items in this section receive a score of zero.

The program runs and reads the input file	_____	30 points
The program displays an error message if the file cannot be found	_____	20 points
The program converts the input to title case	_____	20 points
The program sorts the input into a new ArrayList, in correct order	_____	30 points
The program stops reading the input file when “STOP” is found	_____	20 points
The program displays the sorted list, in correct order	_____	20 points

Source Code Review

The source code is properly commented		
A comment containing the contestant number is present	_____	10 points
Methods and code sections are commented	_____	20 points
A method exists to perform the insert into the ArrayList	_____	20 points
A method exists to perform the compareTo	_____	20 points
A method exists to perform the display of output	_____	20 points
Code uses try... catch for exception handling	_____	20 points
Code uses a consistent variable naming convention	_____	10 points

Total Points = 305

Suggested Solution

```

/**
 * BPA Java Programming Contest : Team Directory
 *
 * @author - contestant number goes here
 *
 * @version January 2018
 */

import java.util.ArrayList;
import java.util.Scanner;
import java.io.*;

public class Sort {
    public static void main(String[] args) throws FileNotFoundException {
        // Variable declarations
        String tempID;
        String tempName;
        ArrayList<TeamMember> list = new ArrayList<TeamMember>();

        try {            // Try to open the input file named in the command line
            Scanner scan = new Scanner(new File("team.txt"));

            tempName = scan.nextLine();    // Reads the file and stores the value in the variable

            // Loops through the file, reading each line, until it reads STOP
            while (!tempName.toUpperCase().equals("STOP"))
            {
                tempID = scan.nextLine(); // Reads the file and stores the value in the variable
                TeamMember tempMember = new TeamMember(tempName, tempID);
                insert(list, tempMember); // calls the insert method
                tempName = scan.nextLine();
            }
            System.out.println(list.toString());    // calls the toString method to output the list
        } catch (FileNotFoundException e) {        // if the file is not found, display an error message
            System.out.println("The input file was not found");
        }
    }

    // This method inserts and sorts the list into the ArrayList
    public static void insert(ArrayList<TeamMember> a, TeamMember s) {
        int mid = 0;
        int len = a.size();
        for (int i = 0; i < len; i++) {
            if (a.get(i).compareTo(s) >= 0) {
                a.add(i, s);
                mid = i;
                break;
            }
        }
    }
}

```

```

        if (mid == 0) a.add(s);
    }
}

```

```

public class TeamMember {
    private String fullName;
    private String idString;

    public TeamMember(String name, String id) {
        idString = id;
        String tempName = "";
        String lastChar = "";
        for (int i = 0; i < name.length(); i++) {
            if (lastChar.equals("") || lastChar.equals("\t")) {
                tempName += name.substring(i, i + 1).toUpperCase();
            } else {
                tempName += name.substring(i, i + 1).toLowerCase();
            }
            lastChar = name.substring(i, i + 1);
        }
        fullName = tempName;
    }

    // Method to return the correct full name
    public String toString() {
        return fullName;
    }

    // Method to compare the strings to each other for sorting
    public int compareTo(TeamMember other) {
        return idString.compareTo(other.idString);
    }
}

```